ARCHITECTURE, CLEANUP, AND TRAINING GUIDE

sniffy

A practical guide to the cleaned codebase, its two-stage network-flow classifier, and your next step: training the attack-type model.

Branch: `decide_if_i_should_change`
Prepared: 2026-10-02
Schema: 46 canonical columns

> **Current state:** the package, capture path, and both trainer classes are implemented. The full tests pass. Stage two has not been trained on your labeled dataset yet; the runbook in this guide takes you through that task.

This report is a learning guide, not a claim that a production model has already been trained or evaluated.

1. What the project does

sniffy converts network traffic into flow-level features, then uses a two-stage XGBoost classifier. Stage one answers *benign or attack?* Stage two runs for predicted attacks and answers *which attack type?*

1. **Capture:** NFStream reads a PCAP or a live interface. Canonical plugins add DNS, ICMP, TTL, packet-size, and TCP-window measurements.
2. **Serialize:** `FlowStreamProcessor` turns each expired flow into a CSV row in canonical schema order.
3. **Prepare:** CSV splitting bounds memory; attack segregation copies rows with `Label == 1` for stage two.
4. **Train:** each trainer cleans features, makes a train/test split per chunk, incrementally fits an XGBoost model, records metrics, and writes its own artifact bundle.
5. **Predict:** the predictor sends every row through stage one, then sends only predicted attacks through stage two.

> **Ground truth matters:** capture does not know whether traffic is malicious. Use labeled datasets for supervised training. A capture row with blank `Label` or `Attack` is not ready for training until it has been labeled.

2. Clean project structure

```
sniffy/
|-- README.md
|-- docs/
|   |-- sniffy_project_guide.pdf
|   `-- sniffy_project_guide.html  # editable source for the guide
|-- pyproject.toml
|-- requirements.txt
|-- capture.py                     # compatibility entrypoint
|-- sniffy/                        # installable package; canonical logic
|   |-- config.py                  # portable paths and model defaults
|   |-- schema.py                  # one ordered 46-column schema
|   |-- data/
|   |   |-- loader.py              # recursive scanner and chunk splitter
|   |   |-- preprocessing.py       # shared feature cleaning and split
|   |   `-- segregation.py         # Label==1 dataset creation and verification
|   |-- models/
|   |   |-- base_trainer.py        # incremental fit, metrics, artifact bundle
|   |   |-- stage1_trainer.py      # binary Label classifier
|   |   |-- stage2_trainer.py      # multiclass Attack classifier
|   |   `-- predictor.py           # stage-one -> stage-two cascade
|   |-- capture/
|   |   |-- live_capture.py        # lazy PyShark/NFStream wrappers
|   |   |-- stream_processor.py    # flow object -> schema-ordered CSV
|   |   `-- plugins/               # one canonical plugin per feature family
|   `-- viz/eda.py                 # parameterized feature histograms
|-- scripts/                       # thin CLI entrypoints
|-- tests/                         # data, capture serialization, model tests
|-- model/xgb_attack_model.pkl     # preserved legacy single-model artifact
|-- proj/capture.pcap              # preserved sample traffic
|-- proj/test/capture.pcap         # preserved sample traffic
`-- sniffy_project_structure.pdf   # original proposal document
```

The old procedural trainer/capture scripts, duplicated plugin implementations, scratch test scripts, orphaned notebook checkpoints, and their tracked bytecode caches have been removed. PCAPs and the old pickle remain as data/reference; the pickle is **not** interchangeable with the new two-stage artifact bundles.

3. File tour: where to read first

File	Responsibility	Why it matters
`sniffy/schema.py`	Ordered feature names, target names, and stage-specific drop columns.	All CSV production, preprocessing, and model feature selection should agree with this one list.
`sniffy/config.py`	`Paths` derives data/artifact paths from environment variables; `TrainerHyperparameters` holds model defaults.	Use this instead of machine-specific absolute paths.
`sniffy/data/loader.py`	`DirectoryScanner` finds sorted CSVs recursively; `DatasetSplitter` writes chunks in schema order.	Training discovers files here, so filenames and directory layout affect iteration order.
`sniffy/data/preprocessing.py`	Drops metadata/target columns, converts features to numeric, replaces infinities/missing values, clips values, and splits train/test.	Stage-one and stage-two differences are configuration, not copy-pasted logic.
`sniffy/data/segregation.py`	Writes attack-only rows and verifies every written `Label` is 1.	It does not invent or validate attack-type names. Those must already be in `Attack`.
`sniffy/models/base_trainer.py`	Counts classes, loops through CSV chunks, warm-starts XGBoost, calculates metrics, and saves a model/classes/features bundle.	This is the main training control flow shared by both stages.
`sniffy/models/stage1_trainer.py`	Binary logistic objective, AUC evaluation, and negative/positive class weighting.	Target is numeric `Label` with values 0 and 1.
`sniffy/models/stage2_trainer.py`	Multiclass soft-probability objective and class-balanced sample weights.	Target is `Attack`; class names are encoded for XGBoost and saved for decoding later.
`sniffy/models/predictor.py`	Loads both bundles, aligns features to the saved feature order, and cascades predictions.	Stage two is called only for rows stage one marks as attacks.
`sniffy/capture/plugins/*.py`	Feature hooks that store custom values on NFStream's `flow.udps`.	These are the only plugin implementations to extend going forward.
`sniffy/capture/stream_processor.py`	Maps flow attributes/custom values into every schema column and appends rows to CSV.	Missing flow/plugin fields receive defaults; labels are caller-provided.

File	Responsibility	Why it matters
`scripts/*.py`	Argument parsing and composition of package classes.	Keep business logic in `sniffy/`; command modules should remain thin.

4. Your next task: train stage two

Input contract

The source data needs the canonical columns, including numeric `Label` values 0/1 and a nonempty `Attack` class for attack rows. Stage two is trained only on rows where `Label == 1`. There must be at least two distinct attack classes in the training corpus.

Run the pipeline

```
source .venv/bin/activate

# Split a labeled source CSV into bounded-size files.
sniffy-split /path/to/labeled_dataset.csv data/splitted --chunksize 500000

# Optional if you still need the binary detector.
sniffy-train-stage1 data/splitted --artifact artifacts/stage1/xgb_v1.pkl

# Build the attack-only data and verify Label == 1.
sniffy-segregate data/splitted data/attacks

# Train and save the multiclass attack-type model.
sniffy-train-stage2 data/attacks --artifact artifacts/stage2/xgb_v1.pkl

# Use both artifacts after stage one and stage two are trained.
sniffy-predict /path/to/flows.csv \
  --stage1 artifacts/stage1/xgb_v1.pkl \
  --stage2 artifacts/stage2/xgb_v1.pkl \
  --output predictions.csv
```

If stage one is already trained and its artifact is available, you can skip its training command. Stage two training itself does not load stage one; it only needs the attack-only CSVs.

Important chunk limitation

`fit_directory` reads up to 500,000 rows per training chunk. `FeaturePreprocessor.transform` currently requires at least two target classes in every processed chunk. The multiclass classifier also needs the classes represented in its training split. If each source file or chunk contains only one attack type, stage-two training can fail even though the directory as a whole contains multiple types.

Before training, inspect the generated files/chunks. If attack types are grouped, create shuffled/interleaved stage-two chunks that include all required classes, or update the trainer to buffer across files before training. Do not silently label a single attack type as another class.

```
from pathlib import Path
import pandas as pd

for csv_path in sorted(Path("data/attacks").glob("*.csv")):
    print(csv_path.name)
    for chunk_number, chunk in enumerate(
        pd.read_csv(csv_path, usecols=["Attack"], chunksize=500_000)
    ):
        counts = chunk["Attack"].dropna().value_counts()
        print(" ", chunk_number, counts.to_dict())
```

Investigate any chunk with fewer than two classes, missing attack labels, or too few rows for the configured 25% test split. The training code reports per-chunk metrics in the returned trainer object and logs them; the CLI prints those metric dictionaries.

5. Understand the two model stages

Stage	Target	Features excluded	Model / artifact
1: detection	`Label` (0 benign, 1 attack)	Source/destination IP, `Dataset`, `Attack`, and target `Label`	`binary:logistic`; `artifacts/stage1/xgb_v1.pkl`
2: attack type	`Attack` (for attack rows only)	Source/destination IP, `Dataset`, `Label`, and target `Attack`	`multi:softprob`; `artifacts/stage2/xgb_v1.pkl`

The base trainer discovers CSVs in sorted order, computes global class counts from the configured source directory, fits one classifier, then warm-starts its booster for later chunks. Metrics include accuracy, precision, recall, F1, and a confusion matrix; ROC-AUC is populated for the binary stage. Each artifact is a joblib bundle containing the XGBoost estimator, stage-two class labels (when applicable), and exact feature order.

For a prediction row, stage one predicts the binary label. Benign rows return `benign`; positive rows are transformed to stage-two features, classified, and decoded back to the original attack label. If stage-one confidence/recall is poor, stage two never gets a chance to recover that missed attack.

6. Capture and labeling

Capture uses `PacketCapturer.stream_flows`, which constructs `NFStreamer` with the canonical plugin instances. `FlowStreamProcessor.process` consumes expired flows and appends one schema-ordered CSV row per flow. The PyShark packet wrapper is available for packet-level capture; the main flow CSV command uses NFStream because the ML features are flow aggregates.

```
sniffy-capture --interface wlan0 --output data/capture.csv

# Add labels only when they are actually known:
sniffy-capture --interface wlan0 --label 1 --attack scan \\
  --output data/labeled_capture.csv
```

Default paths are rooted at the current directory or `SNIFFY_HOME`. Override dataset, split, attack, artifact, and capture paths with `SNIFFY_DATASET`, `SNIFFY_SPLIT_DIR`, `SNIFFY_ATTACK_DIR`,

`SNIFFY_ARTIFACT_DIR`, and `SNIFFY_CAPTURE_CSV`. Live capture requires an interface that exists and OS-level permissions; PyShark additionally requires system `tshark`.

> **Verified here:** PyShark decoded a packet from `proj/capture.pcap`; NFStreamer processed that PCAP and the writer persisted 31 flows with 46 columns. This verifies the offline PCAP path, not live interface permissions.

7. Tests and troubleshooting

- Run the focused suite: `.venv/bin/python -m unittest discover -s tests -v`.
- **Missing target:** check exact column capitalization: `Label` and `Attack`.
- **No attack rows:** verify the source `Label` values are numeric 1, not strings or a different encoding.
- **Stage two says one class:** check attack labels and per-chunk distributions; there must be multiple distinct attack names.
- **Feature mismatch at prediction:** use the same canonical schema; the predictor aligns to saved feature names and fills missing values with zero, but it cannot correct semantically different features.
- **Capture import works but capture fails:** check interface name, `tshark`, packet-capture privileges, and NFStream's source support.
- **Old pickle:** `model/xgb_attack_model.pkl` predates the new bundles and may contain only one stage. Train the new stage artifacts before using `sniffy-predict`.

The current tests cover schema consistency and capitalization, splitting all chunks, attack-only segregation and verification, flow serialization, and a synthetic two-stage train/predict integration test. They do not establish model quality on your dataset; inspect metrics and class distributions after training.

8. Cleanup and preserved files

Removed obsolete code: the duplicated `model/` trainers/helpers, the old `proj/classes/` plugin copies, old capture entrypoints, scratch files under `proj/test/`, notebook checkpoints, and tracked bytecode caches. Their responsibilities now have one canonical home in `sniffy/`.

Preserved deliberately: `proj/capture.pcap`, `proj/test/capture.pcap`, `model/xgb_attack_model.pkl`, the original proposal `sniffy_project_structure.pdf`, and VS Code settings. The removed scratch scripts and checkpoints had no references from the canonical package, scripts, or tests.

Keep new application logic in `sniffy/`, command wiring in `scripts/`, and behavioral checks in `tests/`. When extending the model, update the canonical schema only once and add a small test that protects the data contract.